

SECURE BY DESIGN

Security built in — not bolted on

How ION's 20 secure-by-design principles shape every line of code and every release.

v0.38.0

Engineering & Assurance Brief

Security is a property of the process

Secure by Design means security is designed into the product and its lifecycle from the start — a continuous responsibility across plan, build and operate, not a test bolted on at the end.

ION holds itself to **20 numbered principles**, each with a status that is re-audited **every release** and published in `SECURITY_ASSESSMENT.md`.

Synthesised from established frameworks



NCSC

Secure Design + Secure Development & Deployment



CISA

Secure by Design pledge



NIST SSDF

Secure Software Development Framework



OWASP

Top 10 + ASVS



Saltzer & Schroeder

the classic 8 design principles

Twenty principles across four stages



Govern

P1–P3

- P1 Security is everyone's concern
- P2 Senior accountability
- P3 Radical transparency



Design

P4–P13

- P4 Establish context
- P5 Defence in depth
- P6 Least privilege
- P7 Secure defaults
- P8 Complete mediation
- P9 Economy of mechanism
- P10 Open design
- P11 Don't trust input
- P12 Detect compromise
- P13 Reduce impact



Build & Verify

P14–P18

- P14 Clean, maintainable code
- P15 Secure the repository
- P16 Secure the pipeline
- P17 Eliminate vuln classes
- P18 Continually test security



Operate

P19–P20

- P19 Plan for security flaws
- P20 Continuous risk mgmt

An honest, published posture

19 / 20

principles Met

1

Mostly Met (P1)

0

Partial / Gap

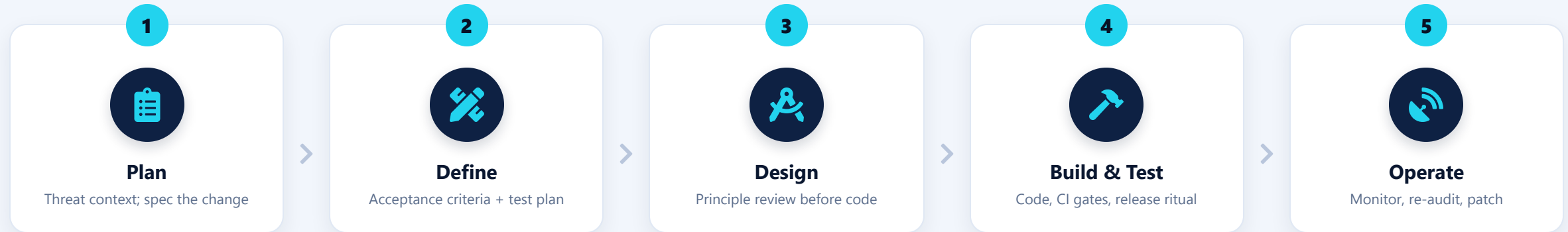
every release

re-audited

The one gap is structural, and we say so. P1 ("security is everyone's concern") is **Mostly Met**: ION is a single-maintainer project, so it cannot reach "Met" without a second human reviewer. Six artifacts — CODEOWNERS, CONTRIBUTING, a security-review agent, and pre-commit gates — bring it as close as the model structurally allows.

No critical, high, medium or low findings outstanding — and the severity-trend table in `SECURITY_ASSESSMENT.md` tracks net-new surface for every version, so the claim is auditable, not asserted.

Five phases, every material change



Secure by Design isn't only for headline features — it applies to every material change, however small.

Principles become engineering rules



P8 · Complete mediation

Ownership & auth checks

Every mutation re-checks ownership **inside the service before it writes** — closing the TOCTOU gap a pin service once had. The rule is pinned in CLAUDE.md and audited.



P6 · Least privilege

RBAC + Focus Mode

Fine-grained resource:action permissions across 10 roles; Focus Mode lets a user shrink to a single active role at a time.



P11 · Don't trust input

Sandbox · params · strict CSP

Jinja SandboxedEnvironment, fully parameterised SQL, and a strict CSP (per-request nonces; script/style-src-attr 'none') — no inline handlers can execute.



P7 · Secure defaults

Fail-safe by default

Capabilities default to the safe state; RAG layers default on but **degrade to a silent no-op** with no AI host — never to an unsafe one.

Designed to contain and reveal



P5 · Defence in depth

Layered middleware

Security-headers, rate-limit, monitoring and request-logging middleware stack; circuit breakers fast-fail every external integration.



P12 · Detect compromise

Tamper-evident ledger

The workbench evidence ledger is a **sha256 hash-chain** — any change is detectable; all actions emit ECS-compliant audit logs.



P13 · Reduce impact

Data minimisation

Session tokens stored only as SHA-256 hashes; configurable retention sweeps; air-gap shrinks the blast radius by construction.



P17 · Eliminate vuln classes

Whole-class removal

Retired python-jose/ecdsa for PyJWT (RS256-only); advisory-lock leader election removes multi-worker race classes outright.

Guardrails that catch issues before review



AI pair-programmer guards

A **PostToolUse ruff** hook lints every edited file; a **PreToolUse** guard gates edits to auth / cert / endpoint paths.



Process artifacts

CODEOWNERS, CONTRIBUTING, and a pre-commit suite (ruff + bandit + pip-audit) mirror the CI gates at the workstation.



Specialist review agents

security-reviewer, workbench-ledger-reviewer (TOCTOU + chain), and release-checker run before commit/tag.



WHY-only comments

Comments capture rationale, not mechanics; design decisions live in durable project memory, not tribal knowledge.



Gotchas pinned by tests

Each subtle bug found (enum-name storage, exclude_unset null-bypass...) gets a regression test so it **cannot silently return**.



Simplicity (P9) + open design (P10)

Server-rendered, no SPA; one obvious way to do things; security never relies on the design being secret.

Nothing ships until the gates are green

pytest

full suite

ruff

lint

bandit

security lint

pip-audit

dependency CVEs

 all four block tag & release



Two-commit pattern

One reviewable feat/fix commit; one mechanical chore(release) commit — clean to audit and bisect.



SBOM in every image

syft generates an SPDX bill of materials at build time, shipped inside the image (P16).



Signed + protected main

Commits are signed; main enforces signature + linear-history branch protection (P15).



Human acceptance walk

Before a release ships, the maintainer drives the **actual built image** through the new feature in a browser.

One ritual, eight files, zero drift

- 1 pyproject.toml — source of truth
- 2 src/ion/__init__.py — template-load-bearing
- 3 README.md — version badge
- 4 Dockerfile — OCI image label
- 5 docker-compose.yml — two service tags
- 6 .env.deploy — ION_VERSION + comment
- 7 CHANGELOG.md — new dated entry
- 8 SECURITY_ASSESSMENT.md — severity column + delta

Why all eight, every time

A single missed bump is a silent integrity bug. `__version__` once rotted for 13 releases — so the ritual is mandatory and two sanity-check greps must return empty before a tag.

Assessment, not afterthought

File 8 forces a security delta **every release**: the severity-trend table extends and any net-new attack surface is named — even when the answer is "0 / 0 / 0 / 0".

Real issues the process caught



v0.20.1

TOCTOU pin ownership

An ownership check ran before, not during, the write. Moved inside the service; now pinned by test and an audit rule.



v0.22.1

Permission null-bypass

A PUT could bypass a perm gate via Pydantic `exclude_unset`. Now compared against stored state via `model_fields_set`.



v0.31.25

Supply-chain typo-squat

The pip-audit gate caught a malicious `fastapi` wheel adding a typo-squatted dep. Pinned away; disclosed in the assessment.



v0.32

Silent enum-name storage

A `SQLEnum` stored the `NAME`, so raw SQL seeds inserted zero rows for 9 versions. Caught, fixed, and pinned by a contract test.

Security isn't a phase. It's the process.

- 20 principles, audited and published every single release
- Designed into the code — least privilege, complete mediation, defence in depth, fail-safe defaults
- Enforced at the gate — CI, SBOM, signed commits, and a human walk of the real image
- Proven by the issues it has caught — and pinned so they never return